PNI ソフトウェア健全性診断レポート

本レポートは、構造化 `.jsonl` ログ解析 CLI ツール `logpulse` (`v0.2.1`) に対して、PNI ステートフル監査ワークフロー `py_cli_audit_tool`（静的 AST 解析・アーキテクチャ構造抽出・隔離コンテナでのオフライン `pytest` / カバレッジ計測・Trivy 脆弱性スキャン）を実行した包括的診断結果である。



logpulse (v0.2.1) PNI 総合健全性スコアカード

診断メタデータ概要

- **対象プロジェクト**: `logpulse` (`v0.2.1` / 対象ルート: `examples/python/cli_audit_tool`)
- **対象言語・ビルド基盤**: `python` (`>=3.11` , `uv` / `uv_build` , CLI フレームワーク `typer` + `rich`)
- **診断ワークフロー / Run ID**: `py_cli_audit_tool` / `wf-7f0bbfdb`
- **評価実施日時**: `2026-10-24T07:35:17.310642+00:00`
- **総合スコア・グレード**: 77 / 100 (Grade B)
- **エンジニアリングルール適合率**: `4 / 15` ルール PASS（ `WARN` : 9件、 `FAIL` : 2件）

Table of Contents

1. エグゼクティブサマリー

2. アーキテクチャとソースコード構成

3. 推奨する変更対応

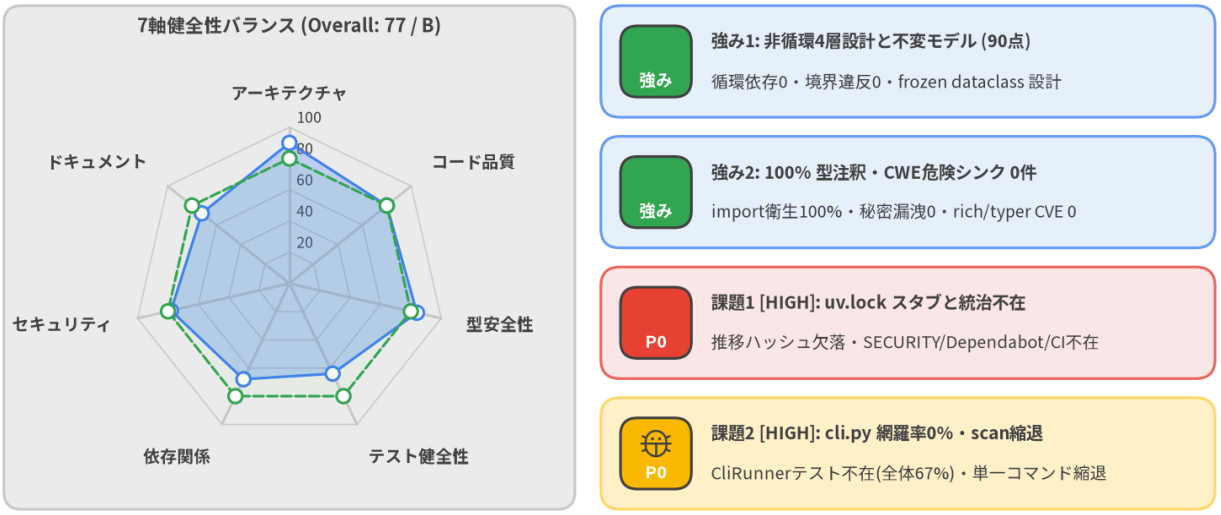
4. 検査指標・ルール適合マトリクスと指摘一覧

5. 詳細分析：コード品質・テスト・動的検証

6. 詳細分析：セキュリティ・脆弱性・API境界

1. エグゼクティブサマリー

`logpulse`（`v0.2.1`）の総合健全性スコアは **77/100 (Grade B)** と評価された。6モジュール・207 LOC のコンパクトで非循環な4層パイプライン設計と 100% の関数型注釈を備え、隔離コンテナでのビルド・オフライン単体テスト・CLI スモーク起動・重大アプリ CVE ゼロを達成している一方、スタブ状態の `uv.lock` と CLI 層のテスト未整備（実測行網羅率 `67.0%`）が最優先の改善課題である。

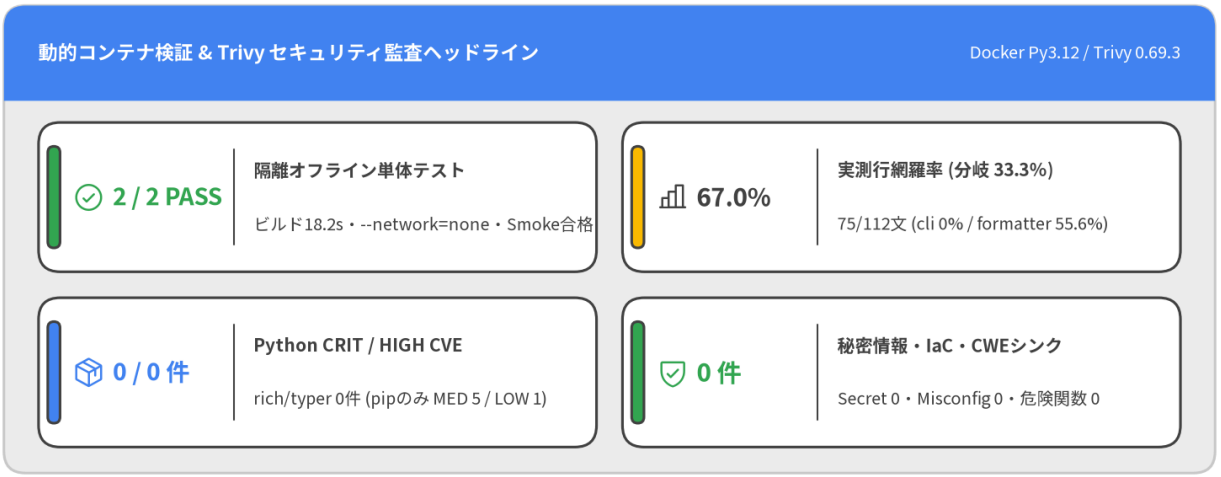


7軸エンジニアリング健全性レーダーと強み・最優先リスク要約

7軸評価サマリーと主要診断結論

- **高評価領域（アーキテクチャ `90`・型安全性 `84`・コード品質 `80`）**：`models.py` を単一の情報源（Single Source of Truth）とし、`parser.py`・`analyzer.py`・`formatter.py` が相互に疎結合な兄弟モジュールとして連なる単方向 DAG を形成している。import 順序衛生・関数シグネチャ型付与率ともに `100.0%` であり、認知的複雑度超過や危険な動的実行イディオムは皆無である。
- **優先改善領域（テスト健全性 `64`・依存関係 `68`・ドキュメント `72`・セキュリティ `78`）**：監査ステップ別では `python_test_audit`（`62/100`）、`repo_structure_audit`（`68/100`）、`container_runtime_audit`（`74/100`）が相対的な弱点となっている。特に `src/logpulse/cli.py` の自動テスト欠如（`0/26` 文）、`uv.lock` の推移パッケージ未記

録、`@app.callback()` 不在による Typer 単一コマンド縮退、および `SECURITY.md`・Dependabot の未整備を優先的に解消すべきである。



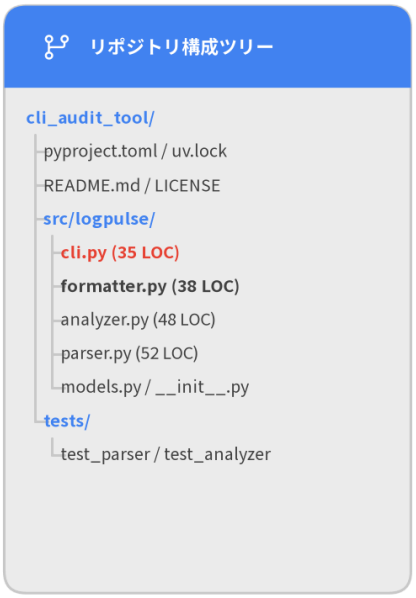
コンテナ動的検証・Trivyセキュリティスキャンの主要ヘッドライン指標

エグゼクティブ結論

- **強み (Verified Strengths)**: PEP 621 準拠の `pyproject.toml` と最小限の実行時依存（`typer`, `rich` の2パッケージ）、不変データクラスによる堅牢なドメイン表現、ならびに `stdout`（データ出力）と `stderr`（エラー・警告出力）の明確なストリーム分離が確立されている。
- **優先対応 (Top Actionable Priorities)**: **P0** として `uv lock` による完全ロックファイル再生成と `typer.testing.CliRunner` を用いた CLI/Formatter テスト拡充を実施し、続く **P1** で `@app.callback()` 追加による `logpulse scan` サブコマンド整合・ファイルストリーム読込化・`SECURITY.md` / Dependabot / CI 整備を完了させることで、Grade A（85点以上）への到達が容易に見込める。

2. アーキテクチャとソースコード構成

`logpulse` は **Layered Pipeline CLI Architecture（CLI アダプタ層 -> 表示整形層・アプリケーション処理層 -> 不変ドメインコア層）** を採用しており、全6ソースファイル（計 `207` LOC・4クラス・8関数）が単方向かつ非循環の依存関係で構成されている。

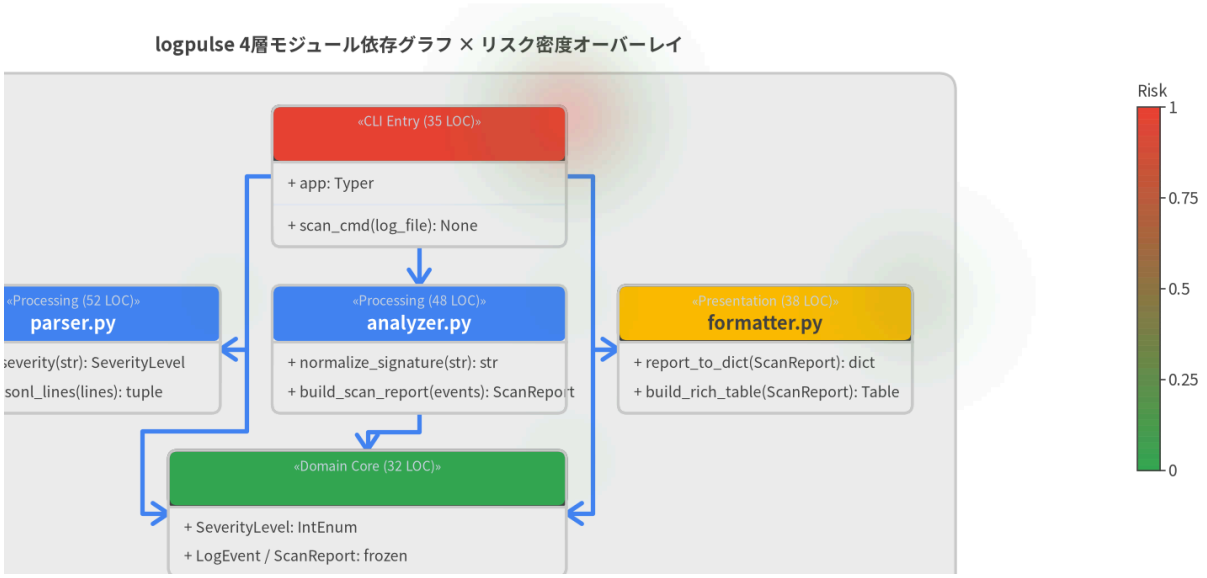


モジュール	LOC	公開型・関数シンボルと主要責務
parser.py	52	parse_severity, parse_jsonl_lines 耐障害JSONL行解析・重要度正規化
analyzer.py	48	normalize_signature, build_scan_report 数値IDの<N>置換・頻度順バースト集計
formatter.py	38	report_to_dict, render_json_report, build_rich_table (JSON / Rich表整形)
cli.py	35	scan_cmd (Typer CLIエントリポイント・ stdout/stderr分離・終了コード制御)
models.py	32	SeverityLevel, LogEvent, SignatureGroup, ScanReport (不変ドメインモデル定義)
__init__.py	2	パッケージ初期化 (future annotations)

ソースディレクトリ構造ツリーとASTモジュールインベントリ (全6ファイル・207 LOC)

レイヤー責務分離と境界健全性・ホットスポット分析

AST の内部 import グラフ検証により、循環依存（`circular_dependencies = 0`）およびレイヤー逆流（`boundary_violations = 0`）が完全にゼロであることが確認された。`models.py` は外部依存を持たず、`parser.py`・`analyzer.py`・`formatter.py` は互いに依存せず `models.py` のみを参照し、最上位の `cli.py` がオーケストレーションを担う。一方、指摘密度に基づくモジュールリスクスコア（`module_hotspots`）では、単体テスト未整備と CLI 境界課題が集中する `src/logpulse/cli.py`（`risk_score=0.89`）および `src/logpulse/formatter.py`（`risk_score=0.27`）が主要なホットスポットとなっている。



4層クラス/モジュール依存関係グラフとリスクホットスポット (HeatmapOverlay)

CLI 実行データフローと状態・エラー処理モデル

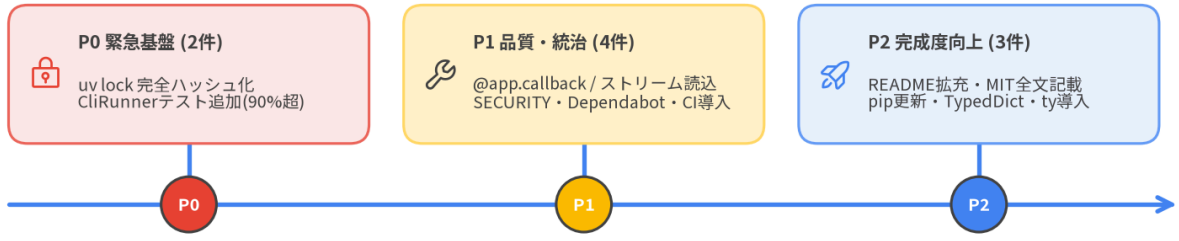
本ツールはデータベースや永続キャッシュを持たないステートレスな単一スレッド実行モデルを採用している。入力ファイルから生成される `LogEvent` ・ `SignatureGroup` ・ `ScanReport` はすべて `@dataclass(frozen=True)` として不変化されており、不正な JSON 行は例外停止させずスキップ行数として集計して `stderr` に警告出力する耐障害設計となっている。



logpulse scan 実行パイプラインと終了コード制御フロー

3. 推奨する変更対応

監査で特定された12件の指摘事項を解消し、総合スコアを Grade B (`77`) から Grade A/S 水準（ `88+` ）へ引き上げるため、緊急度と投資対効果に基づき **P0（緊急基盤整備：2施策） -> P1（品質・統治強化：4施策） -> P2（完成度向上：3施策）** の計9施策を定義する。



優先度フェーズ別 改善ロードマップ (P0 -> P1 -> P2)

フェーズ別実行計画とトレーサビリティ

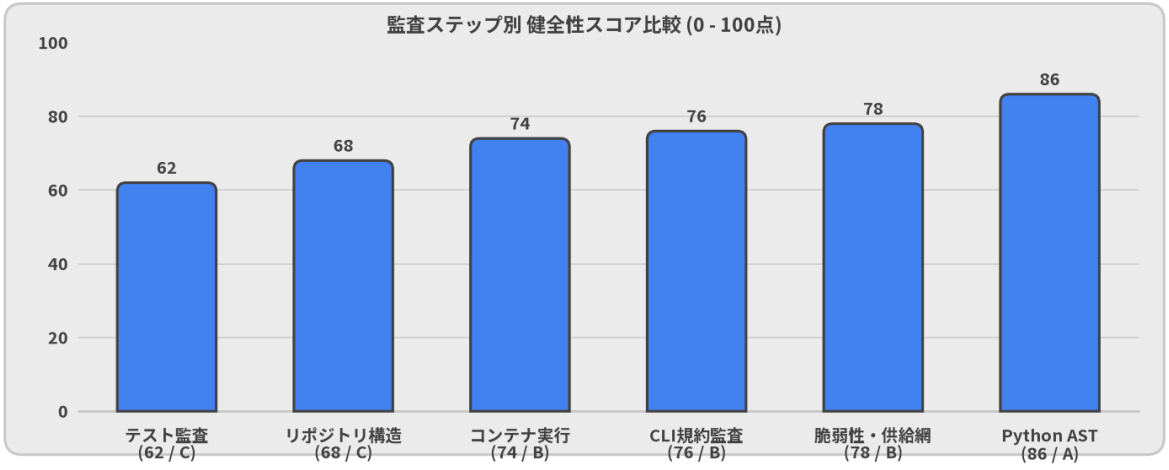
- **Phase 1 (`P0` ー 即時対応)**: 現状12行のスタブに留まる `uv.lock` を `uv lock` で完全解決して静的 SCA の盲点を解消するとともに、 `tests/test_cli.py` （ `typer.testing.CliRunner` ）および `tests/test_formatter.py` を新設し、現在 `0.0%` の `cli.py` カバレッジを一挙に引き上げる。
- **Phase 2 (`P1` ー 短期優先対応)**: `src/logpulse/cli.py` に `@app.callback()` を設けて `README.md` 記載の `logpulse scan <file>` 構文と `--version` / `-h` を両立させ、 `read_text().splitlines()` を逐次ストリーム読込＋ `OSError` / `UnicodeDecodeError` 捕捉（Exit 2）へ置換する。併せて `SECURITY.md` ・ `.github/dependabot.yml` ・最小権限 GitHub Actions CI を整備する。
- **Phase 3 (`P2` ー 継続的品質向上)**: `README.md` と `LICENSE` の標準化、コンテナ配布時の `pip>=26.2.0` 更新・Debian パッチ適用、ならびに `formatter.py` の `TypedDict` 化・Docstring 追加・ `ty` 型チェッカー導入を行う。

優先度	施策	対象ファイル	期待効果	関連ルール
P0	完全な uv.lock 再生成と SHA256 ハッシュ固定	uv.lock pyproject.toml	再現ビルド確立と推移依存 CVE検知の有効化	common/supply_chain python/dependencies
P0	CliRunner 統合テストと Formatter テスト追加	src/logpulse/cli.py src/logpulse/formatter.py	実測行網羅率を 67% から 90%超へ引き上げ	python/testing common/cli_guidelines
P1	@app.callback() 追加と --version / -h 対応	src/logpulse/cli.py README.md	logpulse scan 構文維持と clig.dev 規約準拠	common/cli_guidelines common/documentation
P1	ログ逐次ストリーム読込と CLI境界入力検証	src/logpulse/cli.py	巨大ログOOM防止・読込失敗 Exit 2保証・不正値拒否	common/security common/maintainability
P1	供給網ガバナンス整備 (SECURITY/Dependabot/CI)	. pyproject.toml	OpenSSF 準拠の最小権限CIと 脆弱性自動更新の確立	common/supply_chain
P1	実測行(67%)・分岐(33.3%) 未踏パスの完全解消	src/logpulse/cli.py src/logpulse/parser.py	RV-03 健全基準(行80% / 分岐70%)の確実な達成	common/runtime_verification python/testing
P2	Standard Readme 拡充と MIT ライセンス全文記載	README.md LICENSE	導入・テスト手順・全フラグと ライセンス条項の明確化	common/documentation common/supply_chain
P2	コンテナ pip>=26.2.0 更新と ベースOSパッチ適用	pyproject.toml uv.lock	pip 同梱 6件のCVE解消と Debian OS層リスク低減	common/security_vulnerability CVE-2026-13346 他
P2	Any 排除・Docstring追加・ ty 型検査器導入	src/logpulse/formatter.py pyproject.toml	Docstring 100%達成・Any排除 静的型検査の自動化	python/typing python/code_quality

推奨改善アクション計画マトリクス (P0 -> P1 -> P2 全9施策)

4. 検査指標・ルール適合マトリクスと指摘一覧

本章では、`py_cli_audit_tool` ワークフローで実行した6つの監査ディメンションのスコア比較、評価対象となった全15エンジニアリングルールの適合判定（`FAIL -> WARN -> PASS`）、および重要度順に正規化された全12件の指摘事項（`HIGH -> MEDIUM -> LOW`）を一覧化する。



実行監査ステップ別スコア比較 (弱点順: Weakest -> Strongest)

ルール適合マトリクス（全15ルール：`FAIL: 2` / `WARN: 9` / `PASS: 4`）

全評価ルールは最悪ステータス優先で集約されており、`common/supply_chain`（`uv.lock` スタブおよび `SECURITY.md` / Dependabot / CI 不在）と `python/testing`（CLI・Formatter テスト欠落による網羅率基準未達）の2件が `FAIL` 判定となった。一方で、`common/architecture`・`common/maintainability`・`python/modern_stack`・`common/git_workflow` の4件は違反ゼロで `PASS` を達成している。

判定	ルールID	評価ステップ	決定的エビデンス要約
FAIL	common/supply_chain	repo_structure_audit	SECURITY.md・Dependabot・CI不在、uv.lock 推移ハッシュ欠落
FAIL	python/testing	python_test_audit	テスト2件のみ。cli.py / build_rich_table 未テスト(80%未満)
WARN	common/cli_guidelines	cli_guidelines_audit	--version/-h/stdin不在、@app.callback()不在で scan 縮退
WARN	common/documentation	repo_structure_audit	README に導入・テスト手順・--min-level/--fail-above 未記載
WARN	common/production_readiness	container_runtime_audit	cli.py:34 の read_text() 一括読込で OSError 時トレース露出
WARN	common/runtime_verification	container_runtime_audit	ビルド/テスト/Smoke合格も実測行 67.0%・分岐 33.3%
WARN	common/security	python_ast_audit	秘密情報0件。--min-level 境界検証不足と read_text() 例外未捕捉
WARN	common/security_vulnerability	security_vulnerability_audit	rich/typer CVE 0件・CWE 0件。同梱 pip@25.0.1 に MEDIUM 5件
WARN	python/code_quality	python_ast_audit	Docstring率 88.9%: formatter.py 公開関数2件で Docstring 欠落
WARN	python/dependencies	repo_structure_audit	PEP 621 準拠だが uv.lock が未解決スタブ、dev群に型検査器不在
WARN	python/typing	python_ast_audit	型付与率 100.0% だが report_to_dict が dict[str, Any]
PASS	common/architecture	python_ast_audit	循環依存 0件・境界違反 0件。cli -> 処理/表示層 -> models
PASS	common/git_workflow	repo_structure_audit	.gitignore 整備済み(__pycache__, .venv 等)、不要物混入なし
PASS	common/maintainability	python_ast_audit	全6ファイル 207 LOC(最大52 LOC)、複雑関数・大域状態 0件
PASS	python/modern_stack	python_ast_audit	import衛生 100.0%、pathlib・frozen dataclass・PEP 585 徹底

エンジニアリングルール適合マトリクス (FAIL -> WARN -> PASS 全15ルール)

重要度順 指摘事項一覧（全12件：`CRITICAL: 0` / `HIGH: 2` / `MEDIUM: 6` / `LOW: 4`）

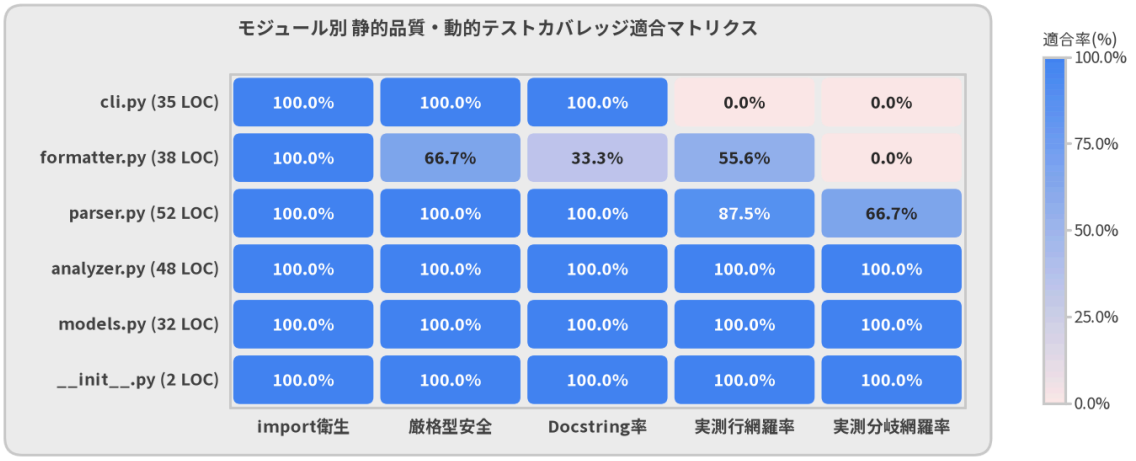
各指摘事項は実在するリポジトリパス・行番号・決定的ファクトに紐付けられている。特に `src/logpulse/cli.py`（4件）と `src/logpulse/formatter.py`（3件）、および供給網設定ファイル群（`uv.lock` / `pyproject.toml` / `LICENSE` / `README.md`）に修正対象が集約されているため、少ない変更範囲で全件を解消可能である。

重要度	ルールID	該当箇所	検出事象・根拠	推奨修正策
HIGH	common/supply_chain	uv.lock:4	rootのみ記載のスタブで 推移版・SHA256欠落	uv lock で完全生成し uv sync --locked 化
HIGH	python/testing	src/logpulse/ cli.py:18	CliRunner テスト 0件で 終了コード未検証	tests/test_cli.py を追加し Exit 0/1/2 を検証
MEDIUM	common/cli_guidelines	src/logpulse/ cli.py:15	@app.callback()不在で scan 縮退・--version欠落	@app.callback() で --version / -h を実装
MEDIUM	common/security	src/logpulse/ cli.py:34	read_text()一括読込・ OSError未捕捉・level未検証	open()逐次読込＋Exit 2捕捉 ＋Enum/Choice 引数化
MEDIUM	common/ security_vulnerability	pyproject.toml:12	同梱 pip@25.0.1 にCVE 6件 Debian OS層に 398件	pip>=26.2.0 更新または除外 ＋ベースOSパッチ適用
MEDIUM	common/supply_chain	.	SECURITY.md・Dependabot・ CI定義が全て不在	脆弱性開示方針・自動更新・ 最小権限CIを追加
MEDIUM	common/ runtime_verification	src/logpulse/ cli.py:18	実測行 67.0%・分岐 33.3% (cli 0% / fmt 55.6%)	cli.py 全26文と formatter 43-50行をテスト追加
MEDIUM	python/testing	src/logpulse/ formatter.py:34	build_rich_table と 非dict行のテスト欠落	test_formatter.py と 境界値テストを追加
LOW	common/supply_chain	LICENSE:1	MIT 見出し2行のみで 許諾・免責全文が欠落	SPDX 標準の MIT ライセンス全文を記載
LOW	common/documentation	README.md:11	導入・テスト手順・一部 CLIフラグが未記載	Standard Readme 構成と 全フラグ表を追記
LOW	python/code_quality	src/logpulse/ formatter.py:13	report_to_dict 等の 公開 Docstring 欠落	Google形式の Args/Returns Docstring を追加
LOW	python/typing	src/logpulse/ formatter.py:13	dict[str, Any] 返却・ py.typed と型検査器不在	TypedDict 定義・py.typed・ ty 型検査器を導入

重要度順 検出指摘一覧 (HIGH -> MEDIUM -> LOW 全12件)

5. 詳細分析：コード品質・テスト・動的検証

本章では、AST 静的解析によって計測されたモジュール単位のコード品質・SIG 保守性・型安全性指標と、隔離 Docker コンテナ（Python 3.12）上で実行された動的ビルド・オフライン `pytest`・`pytest-cov` 実測カバレッジ・CLI スモーク検証の結果を詳細に分析する。



モジュール × 品質ディメンション適合度マトリクス (0 - 100%)

静的コード品質・SIG保守性・型安全性の詳細評価

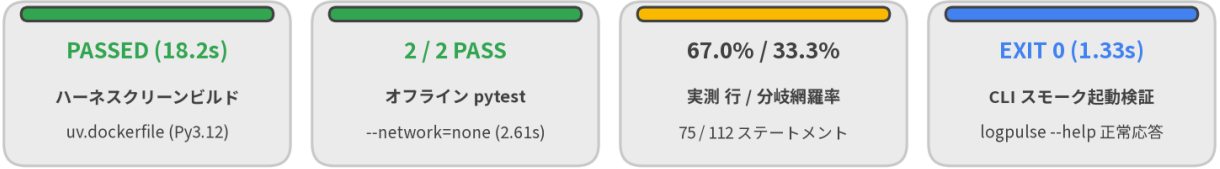
- **SIG 保守性・モジュール結合度**: 全6ファイル合計 `207` LOC（最大は `parser.py` の `52` LOC）と極めて簡潔であり、複雑関数（`complex_functions = 0`）、過剰パラメータ関数（`excessive_parameters = 0`）、可変グローバル状態（`global_state_usages = 0`）、バグリスクイディオム（`bug_risk_idioms = 0`）はいずれもゼロである。
- **型安全性とドキュメンテーション品質**: 全公開関数にパラメータ・戻り値型注釈が付与されており（`type_safety_coverage_pct = 100.0%`）、import 整列率も `100.0%` を達成している。一方、`src/logpulse/formatter.py` の `report_to_dict`（13行目）が `dict[str, Any]` を返却している点、同ファイル13行目・30行目（`render_json_report`）で Google スタイルの Docstring が欠落している点（全体

Docstring 率 `88.9%`）、および PEP 561 `py.typed` マーカーと開発依存での静的型チェッカー（`ty` 等）が未設定である点が改善余地として残る。

- **CLI I/O・境界処理の実運用課題**: `src/logpulse/cli.py` の `scan_cmd`（34行目）は `log_file.read_text(encoding="utf-8").splitlines()` でファイル全体を一度にメモリ展開しており、大容量 `.jsonl` ログでメモリ逼迫を招くほか、読込権限エラーや文字コード不正時に `README.md` で約束された終了コード `2` ではなく未処理例外を送出する。`parser.py` 側は既に `Iterable[str]` を受け取る純粋ストリーム設計となっているため、`log_file.open()` を直接渡す改修が極めて有効である。

コンテナ動的検証（ビルド・オフラインテスト・カバレッジ・Smoke）

PNI の `run_check` によりキャッシュ用ハーネス（`python/uv.dockerfile`, Python 3.12）でクリーンビルド（`18.2s`）を実行し、ネットワーク完全遮断（`--network=none`）環境で `pytest` と `logpulse --help` スモークチェックを実行した結果、すべてエラーゼロで正常完了した。ただし `pytest-cov` による実測カバレッジは行 `67.0%`（`75 / 112` 文）・分岐 `33.3%` であり、`cli.py`（`0.0%`）と `formatter.py` の `build_rich_table`（43〜50行目未実行で `55.6%`）が網羅率を押し下げている。

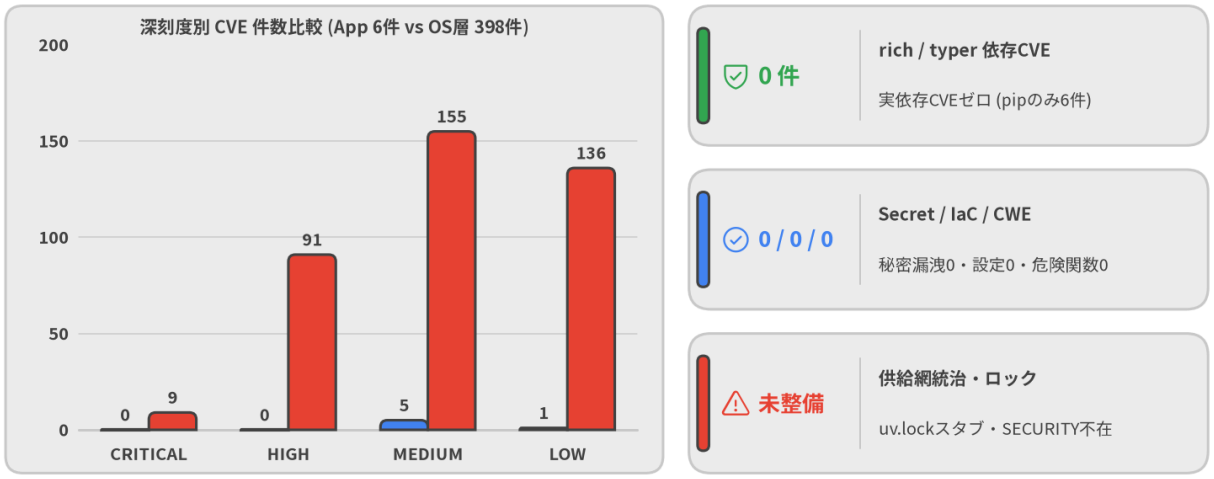


対象モジュール (昇順)	行網羅率	分岐網羅率	実行文 / 総文数	未実行行プレビュー (missing_lines)
src/logpulse/cli.py	0.0%	0.0%	0 / 26	3, 5, 6, 8, 9, 11, 12, 13, 15, 18 (全体未実行)
src/logpulse/formatter.py	55.6%	0.0%	10 / 18	43, 44, 45, 46, 47, 48, 49, 50 (build_rich_table)
src/logpulse/parser.py	87.5%	66.7%	21 / 24	47 (空行スキップ), 54, 55 (非dict JSON行)
src/logpulse/models.py	100.0%	N/A	27 / 27	なし (全データクラス定義・列挙値を網羅)
src/logpulse/analyzer.py	100.0%	100.0%	16 / 16	なし (正規化・フィルタ・ソート全分岐網羅)
src/logpulse/__init__.py	100.0%	N/A	1 / 1	なし

コンテナ動的検証KPIとモジュール別実測カバレッジ内訳 (pytest-cov)

6. 詳細分析：セキュリティ・脆弱性・API境界

本章では、Trivy (`0.69.3`) によるファイルシステムおよび解決済みコンテナイメージに対するSCA（ソフトウェア構成解析）・IaC 設定検査・秘密情報スキャン結果と、AST による危険なCWE コードシンク・API 攻撃面の検査結果を統合して評価する。



アプリケーション依存 vs コンテナOS層 CVE内訳とサプライチェーン統治状況

アプリケーション依存関係 vs コンテナ OS 層の脆弱性分離評価（SEC-01〜SEC-05）

- **アプリケーション依存関係 (`SEC-01` ー `app_vuln_counts`)**: 解決済みコンテナイメージ上の Python パッケージ検査において、 `CRITICAL` および `HIGH` の脆弱性は `0` **件**であった。 `logpulse` が直接・推移的に利用する `rich` ・ `typer` ・ `click` ・ `pygments` 等には脆弱性は一切存在せず、検出された計6件（ `MEDIUM` 5件、 `LOW` 1件）はすべてベースイメージ内のシステム `pip@25.0.1` に起因する（ `pip>=26.2.0` への更新または本番イメージからの `pip` 除外で全件解消可能）。
- **スタブ `uv.lock` による静的ファイルシステムスキャンの盲点**: リポジトリ内の `uv.lock` に対する Trivy FS スキャンでは、ルートパッケージ `logpulse@0.2.1` 1件のみしか認識さ

れなかった。`uv lock` による完全な推移依存バージョンとハッシュの記録が、再現性確保のみならず CI での高速ロックファイル SCA を機能させる上でも不可欠である。

- **OS ベースイメージ層と SAST / Secret / API 境界 (`SEC-02` 〜 `SEC-05`)**: Debian 12.13 ベースレイヤーでは `libgnutls30` (`CVE-2026-42010`) や `openssl` / `libssl3` (`CVE-2026-31789`) を含む計 `398` 件（`CRITICAL` 9件、`HIGH` 91件）が検出されており、コンテナ配布時にはセキュリティ更新適用または Distroless イメージ採用が推奨される。一方、ソースコード自体にはハードコード秘密情報（`0` 件）、IaC 設定不備（`0` 件）、危険な CWE コードシンク（`eval` / `exec` / `shell=True` / `pickle` 等 `0` 件）、外部公開 HTTP API エンドポイント（`0` 件）はいずれも存在せず、コードレベルの攻撃面は極めて小さく保たれている。

区分 / 深刻度	CVE / 検査ID	パッケージ・対象	導入版 -> 修正版	脆弱性・検査結果の概要
App [MEDIUM]	CVE-2026-13346	pip (python-pkg)	25.0.1 -> 26.2.0	悪意あるインデックス経由の 任意ファイル配置
App [MEDIUM]	CVE-2026-8643	pip (python-pkg)	25.0.1 -> 26.1.2	不正 entry point 名による パストラバーサル上書き
App [MEDIUM]	CVE-2026-3219 CVE-2026-6357	pip (python-pkg)	25.0.1 -> 26.1	細工された wheel 展開時の 不適切配置・任意コード実行
App [MED/LOW]	CVE-2025-8869 CVE-2026-1703	pip (python-pkg)	25.0.1 -> 25.3 / 26.0	シンボリックリンク検証不備・ wheel 展開時情報漏洩
OS [CRITICAL]	CVE-2026-42010	libgnutls30 (debian 12.13)	3.7.9-2+deb12u5 -> deb12u7	GnuTLS ユーザ名 NUL 文字による 認証バイパス (CVSS 9.8)
OS [CRITICAL]	CVE-2026-31789	openssl / libssl3 (debian 12.13)	3.0.18-1~deb12u2 -> 3.0.19	大型 X.509 証明書処理における ヒープバッファ溢れ
OS [CRITICAL]	CVE-2026-8376 CVE-2026-42496	perl-base (debian 12.13)	5.36.0-7+deb12u3 -> deb12u4	正規表現コンパイル溢れ・ Archive::Tar パストラバーサル
IaC / Secret	SEC-02 / SEC-03	全リポジトリ対象	検出 0 件 (PASS)	Dockerfile設定不備 0件・ 秘密情報漏洩 0件 (Trivy+AST)
SAST / API	SEC-04 / SEC-05	src/logpulse/*.py	危険シンク 0 件 HTTP API 0 件	CWE-78/89/94/502 危険呼出 0件・ HTTPルート 0件 (CLI専用)

主要検出CVE・修正バージョン対応表およびSAST / Secret / API境界検査サマリー